

Hospital Growth & Automation Operating System

Product Requirements Document (MVP) — Tier 2/3 Indian Hospitals

Alternative positioning: Modern Hospital Operating System for Indian Hospitals

A practical, affordable, mobile-first web app that moves Tier 2 / Tier 3 Indian hospitals from manual chaos (phone calls, WhatsApp, paper registers, receptionist memory) to a simple digital operating system for appointments, leads, doctors, and owner analytics.

Document Control

Field	Value
Product Name	Hospital Growth & Automation Operating System
Short Name	"Hospital OS" (internal codename)
Version	1.0 (MVP PRD)
Status	Ready for build
Owner	Product / Founder
Build Tool	Claude Code
Primary Stack	Next.js + TypeScript + Tailwind + shadcn/ui + Supabase + Vercel
Target Market	Tier 2 / Tier 3 Indian city hospitals & clinics
Last Updated	2026-06-13

How to use this PRD inside Claude Code

1. Paste this document as PRD.md at the repo root.
 2. Build **MVP modules in the phase order** given in Section 39 — do not build everything at once.
 3. Treat **Section 27–30 (database, schema, RLS, API)** as the contract. Frontend follows the data model, not the other way around.
 4. Use **Section 40 (Acceptance Criteria)** as the definition of done for each module.
 5. Keep the **MVP Build Philosophy (Section 10 / 38)** as a guardrail: when in doubt, ship the simpler version.
-

Table of Contents

1. Product Overview

2. Product Vision
3. One-Line Positioning
4. Indian Tier 2 / Tier 3 Market Context
5. Target Hospital Types
6. Target Users
7. Main Problem Statement
8. Business Goals
9. User Personas
10. MVP Scope
11. Non-MVP / Future Scope
12. User Roles and Permissions
13. Complete User Journey
14. Core Feature Modules
15. Page-by-Page Website Structure
16. Admin Panel Structure
17. Appointment Booking System
18. Receptionist Approval System
19. Doctor Dashboard
20. Patient Dashboard
21. Lead Management CRM
22. Hospital Owner Analytics Dashboard
23. Walk-in Patient Handling
24. OPD Queue / Token Logic
25. Notification System
26. Authentication System
27. Supabase Database Structure
28. Suggested Tables and Fields
29. Row-Level Security Rules
30. API / Backend Logic
31. Frontend Component Structure
32. Mobile Responsiveness Requirements
33. Hindi / Hinglish UX Requirements
34. Local SEO Requirements
35. Google Maps and Local Trust Elements
36. Performance Requirements
37. Security and Privacy Considerations
38. MVP Development Roadmap
39. Phase-Wise Build Plan for Claude Code
40. Acceptance Criteria
41. Edge Cases

- 42. Future Scalability Plan
 - 43. Sales Demo Flow for Hospital Owners
 - 44. Marketing Angle
 - 45. 10-Part Reel Series Alignment
 - 46. Final Developer Checklist
-

1. Product Overview

The Hospital Growth & Automation Operating System is a multi-tenant web application that gives small and mid-sized Indian hospitals a single, simple system to capture patient demand and run their front desk. It combines four things that hospitals today stitch together manually:

- **A conversion-focused hospital website** that turns visitors into appointment requests (not a static brochure).
- **An appointment request + receptionist approval workflow** that fits how Indian OPDs actually run.
- **Role-specific dashboards** for the receptionist, the doctor, the owner, and the patient.
- **A lead CRM + owner analytics layer** so no inquiry is lost and the owner can see daily performance.

It is deliberately **not** a hospital ERP, EHR, or billing suite. It targets the highest-pain, lowest-effort wins: stop missing patients, stop losing inquiries, and give the owner visibility. Everything is built on Supabase (Postgres, Auth, RLS, Storage) and deployed on Vercel so it is cheap to run and fast to ship.

The product is designed to be sold and operated by a small agency or solo founder managing many hospitals (multi-tenant), while each hospital experiences it as their own branded system.

2. Product Vision

Build the default digital front desk for every small and mid-sized hospital in Bharat.

In three years, a hospital owner in Kanpur, Gorakhpur, or Kota should be able to sign up, get a fast bilingual website, and within a week have every phone inquiry, WhatsApp message, and walk-in flowing into one screen — with the doctor seeing their day on a phone and the owner seeing growth numbers every morning. The product wins not by having the most features, but by being the **simplest system a non-technical receptionist will actually use every day**.

Think big and scalable (one platform, thousands of hospitals, clean data model, multi-tenant from day one), but execute with Indian jugaad: minimum moving parts, no feature the staff won't use, mobile-first, ship the MVP in weeks, add WhatsApp/SMS/payments only after the core habit is formed.

3. One-Line Positioning

"Turn your hospital's phone calls, WhatsApp messages, and paper registers into one simple screen that books appointments, never loses an inquiry, and shows you your growth — built for Tier 2 and Tier 3 hospitals."

Shorter sales version: **"A modern operating system for your hospital's front desk."**

4. Indian Tier 2 / Tier 3 Market Context

This product is designed around how hospitals in cities like Kanpur, Lucknow, Gorakhpur, Varanasi, Prayagraj, Bareilly, Jhansi, Agra, Patna, Ranchi, Indore, Bhopal, Jaipur, Kota, Gwalior, and Meerut actually operate. The context that shapes every design decision:

- **Phone-first demand.** Most patients call the reception number or send a WhatsApp. The reception phone is often busy during OPD hours, so calls are missed and patients call a competitor.
- **Paper and memory.** Appointments live in a physical register and in the receptionist's head. When she is on leave or busy, the system breaks.
- **Walk-in heavy.** A large share of patients simply arrive. Online and walk-in patients must be merged into one queue.
- **Changing doctor schedules.** Doctors are often visiting consultants with unpredictable availability, so appointments cannot be auto-confirmed — a human must approve.
- **Low staff tech comfort.** Front-desk staff are not power users. UI must be obvious, forgiving, and in language they are comfortable with (Hindi/Hinglish).
- **Mobile and low bandwidth.** Owners and doctors check things on a phone, often on patchy 4G. Pages must be light and fast.
- **Trust is local.** Patients trust Google Maps reviews, OPD timings, doctor photos, and a working "Call Now" button more than fancy design.
- **Price sensitivity.** Hospitals will not pay enterprise SaaS prices. The product must be cheap to run (Supabase free/low tier + Vercel) and priced as an affordable monthly subscription.
- **Hindi/Hinglish preference.** Patient-facing copy and key receptionist labels should be available in Hindi/Hinglish; the dashboard can stay in simple English.

Design implication: **simplicity, speed, language, and reliability beat features.** Every screen should pass the "can a 45-year-old receptionist who has never used SaaS use this on day one?" test.

5. Target Hospital Types

The product must work, with the same core, for all of these:

- Small hospitals (10–50 beds)
- Mid-sized hospitals (50–150 beds)
- Multi-speciality clinics
- Homeopathy clinics
- Dental clinics
- Eye hospitals
- Skin / dermatology clinics
- Orthopedic clinics
- Gynecology / maternity clinics & nursing homes
- Diagnostic centers that also offer doctor consultation
- Local nursing homes
- Doctor-owned single-OPD centers

Design implication: the data model is **doctor + department + appointment** centric, not bed/ward/IPD centric. Departments and services are configurable per hospital so a single dental clinic and a multi-speciality hospital both fit.

6. Target Users

User	Where they work	Primary device	What they want
Hospital owner / admin	Owens or manages the hospital	Mobile	Visibility, growth, control, fewer lost patients
Receptionist / front desk	At the front desk	Desktop + mobile	One simple screen to handle all requests fast
Doctor	OPD room / on the move	Mobile	A clean view of today's patients, nothing else
Patient / attendant	Anywhere	Mobile	Book fast, know the status without calling again
Super admin (SaaS owner / agency)	Runs the platform	Desktop	Onboard hospitals, manage tenants, monitor usage

7. Main Problem Statement

Small and mid-sized Indian hospitals lose patients and run blind because their front desk is manual.

Concretely, the product exists to kill these recurring failures:

- Reception phone always busy → missed calls → patient goes to a competitor.
- Patients call multiple times just to confirm an appointment.
- Walk-in patients and online patients are tracked in different places (or nowhere).
- WhatsApp messages get buried and forgotten.
- Total dependency on a paper register and the receptionist's memory.
- No doctor-wise visibility of appointments.
- No department-wise tracking.
- No structured follow-up; follow-up patients are forgotten.
- No record of lost inquiries — the owner never learns how many patients slipped away.
- The owner has no idea about daily performance.
- Old websites are brochures: no booking, no trust, poor mobile experience, no clear CTA.
- Receptionist is overloaded during OPD hours.
- Doctor schedules change constantly and nothing reflects it.

Outcome we sell: fewer missed patients, less reception chaos, clear visibility for doctor and owner, and measurable growth — without asking staff to learn complex software.

8. Business Goals

For the hospital (the customer's outcomes):

- Capture every inquiry (call, WhatsApp, walk-in, website) in one place — zero lost leads.
- Increase confirmed appointments per week by reducing missed calls and response delay.
- Cut repeat "is my appointment confirmed?" calls by giving patients a status page.
- Give the owner a daily, mobile-friendly view of requests, conversions, and growth.
- Reduce dependency on any single staff member's memory.

For the SaaS business (our outcomes):

- Low cost to build and run (Supabase + Vercel, single codebase, multi-tenant).
- Fast MVP so we can demo to hospital owners within weeks.
- A product that demos in 5 minutes and sells on outcomes, not features.
- Recurring monthly subscription per hospital; expansion via WhatsApp/SMS/payment add-ons later.

- A clean multi-tenant data model so onboarding a new hospital is configuration, not code.

Success metrics (MVP):

- Time to book an appointment request as a patient: under 30 seconds.
 - Receptionist can approve/reschedule a request in under 10 seconds.
 - 100% of inquiries (online + walk-in) stored as leads.
 - Owner opens analytics at least 3 days/week.
 - Onboarding a new hospital (profile + doctors + departments live): under 1 day.
-

9. User Personas

Dr. Rakesh Verma — Hospital Owner (52), Kanpur. Runs a 60-bed multi-speciality hospital. Busy, checks his phone between rounds. Doesn't trust dashboards he can't understand in 10 seconds. Wants to know: how many patients came today, how many we lost, which doctor and department are busiest, and whether his money on Instagram/Google is working. Will pay if it visibly brings more patients.

Pooja — Receptionist (28), Lucknow. Handles the front desk during a 200-patient OPD day. Juggles a ringing phone, a WhatsApp number, a paper register, and walk-ins standing at the counter. Not a software person. Needs one screen, big buttons, fast search by phone number, and labels she understands (Hindi/Hinglish). If the tool slows her down even slightly, she abandons it.

Dr. Asthana — Visiting Consultant (45), Gorakhpur. Comes to the hospital three days a week. Wants only one thing on his phone: who is on his list today, in what order, with their complaint. No clutter, no training. Occasionally adds a one-line note or marks a follow-up.

Sunita Devi — Patient's son booking for his mother (30), Prayagraj. Found the hospital on Google Maps on his phone. Wants to book an OPD slot for his mother without calling five times. Prefers Hindi/Hinglish, low patience for forms, wants to know the appointment is confirmed and when to arrive.

Aman — Super Admin / Agency owner (26), runs the SaaS. Onboards hospitals, sets up their website content, doctors, and departments, and monitors that each tenant is active. Needs a control panel to create hospitals and accounts and to keep tenants isolated.

10. MVP Scope

The MVP must include (and nothing heavier):

1. **Modern hospital website** — mobile-first, conversion-focused, local-trust elements, clear CTAs (Call / Book / WhatsApp / Directions).
2. **Appointment request form** — department, doctor, type, date, preferred time, patient details, symptoms; submits as a *request* (not auto-confirmed).

3. **Receptionist dashboard** — view requests; approve / reject / reschedule; assign doctor & department; status management; search & filter; add walk-in; internal notes.
4. **Doctor dashboard** — today / tomorrow / pending / arrived / completed; patient basics + complaint; short note; mark completed; request follow-up.
5. **Patient status view** — check status, date/time, doctor, department, location, contact; request cancel/reschedule; arrival instructions. Simple phone-number-based access.
6. **Lead CRM** — every inquiry stored with source, status, follow-up date, notes, assignment.
7. **Owner analytics dashboard** — simple KPIs and source-wise leads, doctor/department breakdowns, daily/weekly/monthly counts, conversion, lost leads.
8. **Admin / owner management panel** — manage hospital profile, doctors, departments, services, OPD timings, slots, staff accounts, website content, contact numbers.
9. **Basic OPD / walk-in entry + simple token** — mark arrived, generate token, move to In Consultation, mark completed.
10. **Basic notifications** — in-dashboard notifications + email to hospital + the patient status page. (No WhatsApp/SMS yet.)

Cross-cutting MVP requirements: multi-tenant (hospital-scoped data), role-based access with Supabase RLS, mobile-first, Hindi/Hinglish on patient-facing surfaces, fast and light pages.

11. Non-MVP / Future Scope

Explicitly NOT in MVP (do not build yet):

- Full EHR / patient medical history records
- Digital prescription generation
- Lab reports upload and management
- Payment gateway / online payments / UPI
- Teleconsultation / video
- Native mobile apps (patient or doctor)
- AI assistant / AI receptionist
- Complex insurance workflows
- Full billing / invoicing system
- Multi-hospital enterprise features (beyond basic multi-tenant) and multi-branch

Future scope (Phase 2+, see Section 42):

- WhatsApp bot + WhatsApp notifications
- SMS reminders and automated follow-up reminders
- Missed-call to CRM integration
- Online payments / UPI

- Teleconsultation
 - Lab report upload, digital prescription, patient history
 - Doctor mobile app, patient mobile app
 - Multi-branch support
 - Review collection system, referral tracking
 - Staff attendance, inventory, pharmacy module
-

12. User Roles and Permissions

Six logical roles. Five are auth roles; "Walk-in patient entry" is an action performed by the receptionist, not a login.

Role	Logs in?	Scope	Can see patient PII?
Super Admin	Yes	All hospitals (platform)	Limited; for support/ops only
Hospital Owner / Admin	Yes	One hospital (full)	Yes, within their hospital
Receptionist	Yes	One hospital (front desk)	Yes, within their hospital
Doctor	Yes	Own appointments only	Limited (only assigned patients, only needed fields)
Patient	Light (phone-based)	Own appointments only	Own data only
Walk-in entry	N/A (receptionist action)	—	—

High-level permission matrix (✓ = allowed):

Capability	Super Admin	Owner/ Admin	Receptionist	Doctor	Patient
Manage hospitals (create tenants)	✓	—	—	—	—
Manage hospital profile & content	✓	✓	—	—	—
Manage doctors / department	✓	✓	—	—	—

s / services					
Manage staff accounts	✓	✓	—	—	—
View all appointments (hospital)	✓	✓	✓	—	—
Approve / reject / reschedule appts	—	✓	✓	—	—
Add walk-in / manage queue & token	—	✓	✓	—	—
View own appointments	—	✓	✓	✓	✓
Add consultation note / mark completed	—	—	—	✓	—
Request cancel / reschedule (own)	—	—	—	—	✓
View analytics	✓	✓	partial	—	—
View full lead CRM	✓	✓	✓	—	—

Detailed access rules are enforced in Supabase RLS — see Section 29.

13. Complete User Journey

End-to-end flow (the product's spine):

Patient

- Mobile-friendly hospital website (finds hospital via Google/Maps/Instagram)
- Taps "Book Appointment"
- Fills 30-second request form (dept, doctor, date, time, name, phone, problem)
- Submits → creates an APPOINTMENT REQUEST (status: Pending) + a LEAD
- Receptionist sees the new request on her dashboard (in-app + email alert)

- Receptionist Approves / Reschedules / Rejects, assigns doctor + slot
- Patient gets a status update (status page; future: WhatsApp/SMS)
- On the day: patient arrives → Receptionist marks Arrived + issues Token
- Doctor sees the patient on the Doctor Dashboard queue
- Doctor marks In Consultation → Completed, optionally Follow-up Required
- Lead status auto-updates (Appointment Booked → Visited → Converted)
- Owner Analytics aggregates everything (requests, conversions, sources, growth)
- Hospital grows: fewer missed patients, better follow-up, clear visibility

Parallel walk-in journey:

Walk-in patient arrives at counter

- Receptionist taps "Add Walk-in", enters name + phone + dept/doctor
- System creates patient + appointment (status: Arrived) + lead (source: Walk-in)
- Token issued → enters same OPD queue as online patients
- Doctor sees them in the same list → consults → Completed

This single, unified flow is the core demo and the core promise: **one screen for online + walk-in, from inquiry to growth.**

14. Core Feature Modules

The MVP is composed of ten modules. Each maps to a part of the reel series (Section 45) so the product is easy to demo and sell.

#	Module	Core promise	Reel
1	Modern Hospital Website	A site that converts visitors into appointment requests	Part 8
2	Smart Appointment Booking	Book a request in 30 seconds	Part 2
3	Receptionist Dashboard	One screen to approve/reject/reschedule	Part 3
4	Doctor Dashboard	Doctor sees today/tomorrow/pending/arrived	Part 4
5	Patient Dashboard	Patient checks status without calling	Part 5
6	Lead Management CRM	Every inquiry stored with status	Part 6

7	Owner Analytics	& follow-up Owner sees KPIs, sources, and growth	Part 7
8	OPD Queue / Token	Simple arrived → token → consult → done	Part 3/10
9	Admin / Owner Panel	Manage hospital, doctors, departments, content	Part 10
10	Notifications	In-app + email + status page (WhatsApp later)	Part 9

Sections 15–25 specify each module. Sections 27–31 specify the data and code structure that backs them.

15. Page-by-Page Website Structure (Module 1: Modern Hospital Website)

A premium, mobile-first, local-trust-focused public website per hospital. It must **convert**, not just inform. Built with Next.js App Router, server-rendered for SEO and speed.

Public pages (routes):

Route	Page	Purpose	Must include
/	Home	Convert visitor → booking	Hero with H1 (hospital + city), primary CTA "Book Appointment", Call Now, WhatsApp, OPD timings, trust badges, featured doctors, departments, reviews, map, emergency number sticky
/about	About Hospital	Build trust	Story, accreditations, photos, years of service, owner/doctor

/doctors	Doctors	Show expertise	message Doctor cards (photo, name, qualification, department, OPD days), "Book with this doctor" CTA
/doctors/[slug]	Doctor detail	Convert on a specific doctor	Bio, timings, department, book CTA
/departments	Departments	Navigate by need	Department cards with icon + short description + book CTA
/departments/[slug]	Department detail	Department-level SEO + booking	Services, doctors in dept, CTA
/services	Services	List offerings	Service cards, prices optional, CTA
/book	Appointment Booking	The conversion engine	Full request form (Section 17)
/emergency	Emergency Contact	Urgent path	Big emergency number, ambulance, 24x7 note, one-tap call
/location	Location / Maps	Local trust + directions	Embedded Google Map, address, landmarks, "Get Directions"
/contact	Contact	Reach the hospital	Phone, WhatsApp, address, hours, simple message form (creates a lead)
/faq	FAQ	Reduce repeat calls	OPD timing, fees, documents, insurance, parking, etc.
/reviews	Reviews / Testimonials	Social proof	Patient testimonials, Google rating, photos

/status	Patient status	Self-service status	Phone-based lookup (Section 20)
/login	Login hub	Staff access	Patient / Doctor / Receptionist / Admin login entry points

Global website elements (every page):

- Sticky header: hospital logo + name + **Call Now** + **Book Appointment**.
- Floating WhatsApp button (uses <https://wa.me/<number>?text=...> prefilled in Hindi/Hinglish).
- Sticky mobile bottom bar: **Call** · **WhatsApp** · **Book** · **Directions**.
- Footer: OPD timings, emergency number, address, map link, quick links, social links.
- Trust badges: years of service, number of doctors, departments, Google rating, "Govt. registered" where applicable.

Conversion rules: A booking CTA must be visible within one thumb-reach on every screen; Call Now must be one tap; no page should require more than 2 taps to start booking.

16. Admin Panel Structure (Module 9)

The admin/owner panel is a protected area under /admin. It is where the hospital is configured and run at the management level.

Admin sections (routes under /admin):

Route	Section	Who	Purpose
/admin	Dashboard	Owner/Admin	KPI snapshot + quick links
/admin/profile	Hospital Profile	Owner/Admin	Name, logo, address, phone, WhatsApp, emergency, Google Maps link, about
/admin/doctors	Doctors	Owner/Admin	CRUD doctors, photos, qualifications, departments, OPD days/timings, slug, active toggle
/admin/departments	Departments	Owner/Admin	CRUD departments + slugs + icons

/admin/services	Services	Owner/Admin	CRUD services + optional price
/admin/timings	OPD Timings & Slots	Owner/Admin	Per-doctor OPD timings, holidays, slot config
/admin/staff	Staff Accounts	Owner/Admin	Create receptionist & doctor accounts (invite/credentials)
/admin/appointments	All Appointments	Owner/Admin	Read all, filter, override status
/admin/leads	Lead CRM	Owner/Admin	Full CRM (Section 21)
/admin/analytics	Analytics	Owner/Admin	Full analytics (Section 22)
/admin/content	Website Content	Owner/Admin	Edit homepage hero, about, testimonials, FAQ, banners
/admin/settings	Settings	Owner/Admin	Contact numbers, language default, notification email, branding

Super Admin area (/super): create/disable hospitals (tenants), create owner accounts, view tenant usage, impersonate for support. Strictly isolated from hospital-scoped data via RLS.

Admin UX principles: left nav + content; everything reachable in ≤ 2 clicks; mobile-friendly (owner uses a phone); destructive actions confirmed; every change writes an audit log entry (Section 28).

17. Appointment Booking System (Module 2)

Goal: a patient (or attendant) submits a complete, useful appointment **request** in under 30 seconds. It is a request, **not** an auto-confirmed booking — because Indian OPD schedules and walk-in load change constantly and a human must approve.

Form fields (/book):

Field	Type	Required	Notes
Department	select	Yes	From hospital's departments
Doctor	select	No	Filtered by

Appointment type	select	Yes	department; "Any available" allowed OPD / Follow-up / Emergency request
Preferred date	date	Yes	No past dates; respects holidays
Preferred time slot	select	Yes	From available slots or simple ranges (Morning/Evening)
Patient name	text	Yes	
Phone number	tel	Yes	10-digit India; primary key for patient lookup
Age	number	Yes	
Gender	select	Yes	Male / Female / Other
City / area	text	No	Helps local analytics
Problem / symptoms	textarea	No	Short free text; helps doctor & triage
Language preference	toggle	No	Hindi / English (defaults to hospital setting)

Behaviour:

- Multi-tenant: the form is bound to a hospital (subdomain/path or hospital_id in config).
- On submit: create appointments row (status pending) + patients row (create-or-match by phone within hospital) + leads row (source website, status new).
- Validation: React Hook Form + Zod. Inline, friendly, bilingual error messages.
- Confirmation screen: "Aapki request mil gayi hai / We've received your request. We'll confirm shortly." Show a **status link** (/status?phone=... or a short reference id) and a Call/WhatsApp fallback.
- Spam control (MVP-light): basic rate limit per phone/IP, honeypot field; OTP is future scope.
- Accessibility: large tap targets, numeric keypad for phone/age, minimal typing.

Default appointment flow (do not change in MVP):

Patient submits request (Pending)

→ Receptionist reviews

→ Approve (assign doctor + confirmed slot) | Reschedule (propose new slot) |

Reject (with reason)

→ Patient status updates

Appointments are **never auto-confirmed** in MVP.

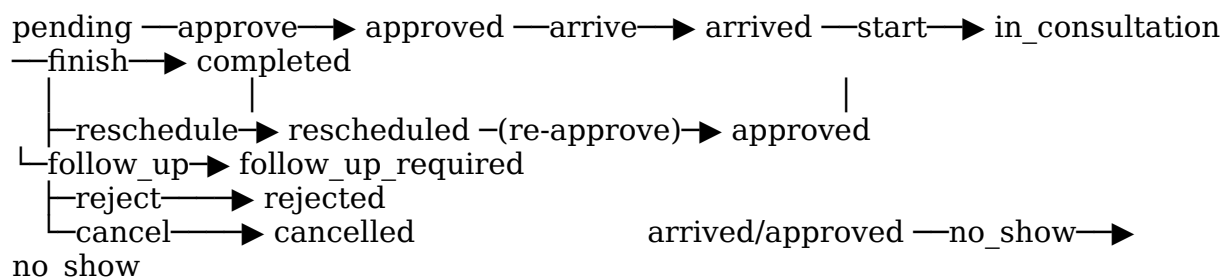
18. Receptionist Approval System (Module 3)

The most-used screen in the product. It must be fast, obvious, and usable by non-technical staff. Route: /reception.

Capabilities:

- View new/pending appointment requests (newest first, live/refreshed).
- Approve, Reject (with reason), or Reschedule (propose new date/time).
- Add a walk-in patient (Section 23).
- Assign / change doctor and department.
- Update appointment status (full lifecycle below).
- Search by patient name or phone number (instant).
- Filter by doctor, department, date, status.
- Add internal notes (not visible to patient).
- Mark patient as Arrived, In Consultation, Completed, No-show.

Appointment statuses (state machine):



Allowed statuses: pending, approved, rescheduled, rejected, arrived, in_consultation, completed, no_show, cancelled, follow_up_required.

UX requirements:

- Card or table list with big, color-coded status badges and big action buttons.
- Each request shows: name, phone (tap to call), age/gender, department, doctor, preferred date/time, problem, source.
- Approve/Reschedule must be doable in ≤2 taps. Reschedule opens a slot picker; Reject asks for a quick reason (preset chips: "Doctor unavailable", "Slot full", "Duplicate", "Other").

- "Today" view is the default; a separate "Requests" (pending) view shows the inbox.
 - Every status change writes to audit_logs and updates the linked lead.
 - Works on desktop (primary) and mobile (counter staff on phone).
-

19. Doctor Dashboard (Module 4)

A clean, fast, distraction-free view for doctors — optimized for mobile. Route: /doctor.

Capabilities:

- See **Today's** appointments (default), **Tomorrow's**, **Pending**, **Arrived**, **Completed**.
- View patient basic details (name, age, gender) and problem/symptoms.
- Add a short consultation note.
- Mark consultation **Completed**.
- Request **Follow-up** (sets a follow-up date → creates/updates lead follow-up).
- Filter by date.

UX requirements:

- The doctor sees **only their own** appointments (enforced by RLS).
 - Default screen = today's queue, ordered by token/arrival. Big, readable, one patient per row.
 - Minimal fields only — no PII beyond what's needed (no full address etc.).
 - One-tap "Mark Completed"; note is optional and short.
 - "Arrived" patients highlighted so the doctor knows who is waiting now.
 - Tomorrow's list helps the doctor plan; pending list is informational.
 - No clutter, no analytics, no CRM — just the day's patients.
-

20. Patient Dashboard / Status View (Module 5)

Self-service status so patients stop calling repeatedly. Route: /status. Authentication kept deliberately light (see Section 26).

Capabilities:

- Check appointment status (Pending / Approved / Rescheduled / Rejected / Completed, etc.).
- View appointment date and time.
- See doctor name and department.
- See hospital location (map link) and contact number.
- Request **cancellation**.
- Request **reschedule**.
- See simple instructions: "Please arrive 15 minutes before your appointment / Kripya apne appointment se 15 minute pehle aayein."

Access model (MVP):

- **Phone-number-based lookup:** patient enters the phone number used to book; system shows their recent appointment(s) for that hospital. Optionally a short reference id shown at booking for a second factor.
- **OTP:** future scope (Phase 2) for stronger verification.
- **Magic link:** optional — emailed/SMS link to the status page (Phase 2).
- **No passwords for patients** in MVP.

UX requirements: bilingual, large status badge, clear next step, Call/WhatsApp/Directions buttons, "request reschedule/cancel" creates a request the receptionist sees (does not directly mutate confirmed schedule without staff action).

21. Lead Management CRM (Module 6)

Every inquiry — from any channel — becomes a lead. Nothing is lost. Routes: /admin/leads (owner) and a focused /reception/leads view.

Lead fields:

Field	Type	Notes
name	text	
phone	text	dedupe key within hospital
city_area	text	
department	fk	preferred department
doctor_preference	fk (nullable)	
source	enum	see below
status	enum	see below
appointment_date	date (nullable)	linked if booked
follow_up_date	date (nullable)	drives follow-up list
notes	text	
assigned_receptionist	fk users	
created_at	timestamp tz	

Lead sources: website, phone_call, whatsapp, google_maps, instagram, facebook, walk_in, referral, other.

Lead statuses: new, contacted, appointment_booked, visited, not_interested, follow_up_required, converted, lost.

Behaviour:

- Created automatically from website booking (website) and walk-in (walk_in); created manually for phone/WhatsApp/Instagram inquiries by the receptionist.
- Status auto-syncs with the linked appointment lifecycle (e.g., appointment completed → lead visited/converted; no_show → keep as follow_up_required or lost).
- Follow-up list = leads where follow_up_date ≤ today and status not closed.
- Owner sees all leads in one place; can filter by source, status, department, date, assigned staff.
- Lost-lead capture: any inquiry that didn't convert is visible (status lost / not_interested) — this is a key owner insight.

UX requirements: table + quick filters; one-tap status change; click-to-call phone; add note inline; "Add Lead" button for manual channels with a source picker.

22. Hospital Owner Analytics Dashboard (Module 7)

Simple, useful KPIs the owner understands in 10 seconds on a phone. Route: /admin/analytics. Charts via Recharts; keep them simple in MVP.

KPIs (cards):

- Total appointment requests (today / this week / this month)
- Approved appointments
- Completed appointments
- Rejected appointments
- No-shows
- Walk-in patients
- Online appointment requests
- Follow-ups pending
- Conversion rate (requests → completed)
- Lost leads

Breakdowns (charts/tables):

- Doctor-wise appointments (bar)
- Department-wise appointments (bar)
- Source-wise leads (pie/bar): website / phone / WhatsApp / Maps / Instagram / Facebook / walk-in / referral
- Daily appointment count (line, last 30 days)
- Weekly growth and Monthly growth (line / % change)
- Most demanded department, Most booked doctor (highlighted stats)

Rules:

- All numbers scoped to the hospital and a date range (default: this month, with today/week/month/custom toggles).
 - Computed from appointments, leads, walk_ins via SQL views / Postgres RPCs (Section 30) — not heavy client computation.
 - Keep it readable: avoid dense dashboards; 6–10 cards + 3–4 charts max in MVP.
 - Mobile-first layout: cards stack; charts are simple and legible on a phone.
-

23. Walk-in Patient Handling (Module: walk-ins, used by Reception)

Walk-ins are first-class. The receptionist adds them in seconds and they enter the same queue as online patients.

Add walk-in flow (/reception → "Add Walk-in"):

- Minimal fields: name, phone, age (optional), gender (optional), department, doctor (optional/any).
- On save: create-or-match patients by phone → create appointments (status arrived, type opd, source walk_in) → create leads (source walk_in) → optionally issue token (Section 24).
- The walk-in immediately appears in the OPD queue and on the assigned doctor's dashboard.

Rules:

- A walk_ins record (or appointment with is_walk_in = true) lets analytics separate walk-in vs online.
 - No website account needed; entirely staff-driven.
 - Same status lifecycle as online appointments from arrived onward.
-

24. OPD Queue / Token Logic (Module 8)

A deliberately simple token/queue system — enough to bring order to OPD hours, not a full queue-management product.

Receptionist can:

- Mark patient as **Arrived** (online approved patient or walk-in).
- **Generate a token number** (simple incrementing number, scoped per hospital per doctor per day).
- See the **waiting list** (all arrived, ordered by token).
- Move a patient to **In Consultation**.
- Mark **Completed**.

Token rules (MVP):

- Token = next integer for (hospital_id, doctor_id, date) starting at 1 each day.

- Generated at arrival; stored on the appointment (token_number).
- Display: "Token #12 — Dr. Asthana — Ortho". Optional simple "now serving" indicator on a queue screen.
- Keep it simple: no priority queues, no multi-counter logic, no SMS-when-near in MVP (future scope).

Doctor view: sees arrived patients in token order; advancing status (In Consultation → Completed) moves the queue.

25. Notification System (Module 10)

MVP keeps notifications cheap and reliable; richer channels are future scope.

MVP notifications:

- **In-dashboard notifications:** a bell/notification list for receptionist and owner — new request, reschedule request, cancellation request, follow-up due. Backed by a notifications table + Supabase Realtime (optional) or polling.
- **Email to hospital:** on a new appointment request, send an email to the hospital's notification address (via Supabase Edge Function + a transactional email provider, e.g., Resend). Keep templates bilingual and short.
- **Patient status page:** the patient's primary "notification" channel in MVP — they check /status.

Future notifications (Phase 2+):

- WhatsApp notifications (confirmation, reminder) via WhatsApp Business API / provider.
- SMS notifications and reminders.
- Automated follow-up reminders (scheduled jobs).
- Missed-call → CRM integration.
- WhatsApp bot for booking and status.

Design note: abstract notifications behind a single notify(event, payload) service so new channels (WhatsApp/SMS) plug in later without touching feature code.

26. Authentication System

Supabase Auth, with role stored on the user and enforced via RLS. Keep staff auth standard and patient auth light.

Roles & login:

- **Super Admin / Owner / Receptionist / Doctor:** Supabase Auth email + password (magic link optional). Role and hospital_id stored in a users/profile table and in JWT

claims (via custom claims / auth.uid() → profile lookup). Accounts are created by an admin (invite or set credentials), not self-signup.

- **Patient: no password** in MVP. Status access via **phone-number lookup** (/status? phone=...), optionally guarded by a short booking reference id. OTP and magic link are future scope for stronger verification.

Rules:

- Single sign-in page hub (/login) routes each role to its dashboard after auth based on role.
- Protect all /admin, /reception, /doctor, /super routes with Next.js middleware + server-side session checks; never rely on client-side checks alone.
- Every authenticated request is scoped to the user's hospital_id (multi-tenant isolation) and role — enforced in RLS, not just UI.
- Session handling via Supabase SSR helpers (cookies) for the App Router.
- Password reset via Supabase; staff onboarding via invite link.

JWT / claims: store role and hospital_id so RLS policies can read them (auth.jwt() -> 'hospital_id' or via a profiles join). Doctors additionally map to a doctor_id for "own appointments only" rules.

27. Supabase Database Structure

Principles:

- **Multi-tenant by hospital_id.** Almost every table carries hospital_id (FK → hospitals.id). RLS isolates tenants.
- **Postgres + RLS first.** Business rules live close to the data; the frontend trusts RLS, not just UI checks.
- **UUID primary keys** (id uuid default gen_random_uuid()), created_at timestamptz default now(), updated_at maintained by trigger.
- **Enums** for statuses/sources (Postgres enum types or text + check).
- **Soft state, hard audit.** Sensitive changes (status, assignments, content) write to audit_logs.
- **Indexes** on every foreign key and on common filters (hospital_id, status, appointment_date, phone).

Enum types:

create type user_role **as** enum

('super_admin', 'owner', 'receptionist', 'doctor', 'patient');

create type appointment_status **as** enum

('pending', 'approved', 'rescheduled', 'rejected', 'arrived', 'in_consultation', 'completed', 'no_show', 'cancelled', 'follow_up_required');

create type appointment_type **as** enum ('opd', 'follow_up', 'emergency');

create type lead_source **as** enum

('website','phone_call','whatsapp','google_maps','instagram','facebook','walk_in','referral','other');

create type lead_status **as** enum

('new','contacted','appointment_booked','visited','not_interested','follow_up_required','converted','lost');

28. Suggested Tables and Fields

For each table: field name, type, purpose, relationships, indexes, and an RLS note. (fk = foreign key.)

hospitals (tenant root)

Field	Type	Purpose / Notes
id	uuid pk	Tenant id
name	text	Hospital name
slug	text unique	Subdomain/path routing & SEO
logo_url	text	From Storage
address	text	Full address
city	text	Tier 2/3 city (analytics, SEO)
phone	text	Primary reception number
whatsapp	text	WhatsApp number (wa.me)
emergency_phone	text	Emergency/ambulance
google_maps_url	text	Embed + directions
about	text	About content
default_language	text	'hi' / 'en' / 'hinglish'
notification_email	text	Where new-request emails go
is_active	boolean	Tenant enable/disable
created_at	timestampz	

Relationships: parent of nearly all tables. Indexes: slug, city. RLS: Super Admin all; Owner/staff read own (id = hospital_id).

users (staff profiles; 1:1 with auth.users)

Field	Type	Purpose / Notes
-------	------	-----------------

id	uuid pk	= auth.users.id
hospital_id	uuid fk → hospitals	Tenant (null for super_admin)
role	user_role	Access role
full_name	text	
email	text	Login
phone	text	
doctor_id	uuid fk → doctors (nullable)	Links a doctor login to its doctor record
is_active	boolean	
created_at	timestampz	

Indexes: hospital_id, role, doctor_id. RLS: user reads self; Owner manages users in own hospital; Super Admin all.

doctors

Field	Type	Purpose / Notes
id	uuid pk	
hospital_id	uuid fk	Tenant
name	text	
slug	text	/doctors/[slug]
qualification	text	e.g., MBBS, MD
department_id	uuid fk → departments	Primary department
photo_url	text	Storage
bio	text	
opd_days	text[] / jsonb	e.g., ["Mon","Wed","Fri"]
opd_timings	jsonb	Per-day start/end
consultation_fee	numeric (nullable)	Optional display
is_active	boolean	Show/hide
created_at	timestampz	

Indexes: hospital_id, department_id, slug. RLS: public read for active doctors of a hospital (website); Owner manages; Doctor reads self.

departments

Field	Type	Purpose / Notes
-------	------	-----------------

id	uuid pk	
hospital_id	uuid fk	Tenant
name	text	e.g., Orthopedics
slug	text	/departments/[slug]
icon	text	Icon key
description	text	
is_active	boolean	
created_at	timestamp tz	

Indexes: hospital_id, slug. RLS: public read (active); Owner manages.

services

Field	Type	Purpose / Notes
id	uuid pk	
hospital_id	uuid fk	Tenant
department_id	uuid fk (nullable)	Optional grouping
name	text	e.g., X-Ray, Root Canal
description	text	
price	numeric (nullable)	Optional
is_active	boolean	
created_at	timestamp tz	

Indexes: hospital_id, department_id. RLS: public read (active); Owner manages.

patients

Field	Type	Purpose / Notes
id	uuid pk	
hospital_id	uuid fk	Tenant; patient identity is per-hospital
name	text	
phone	text	Dedupe key within hospital
age	int	
gender	text	Male/Female/Other
city_area	text	

created_at timestamptz

Indexes: unique (hospital_id, phone), hospital_id. RLS: staff of hospital read/write; patient reads own (matched by phone via status flow); doctor sees limited fields via appointment join only.

appointments (core table)

Field	Type	Purpose / Notes
id	uuid pk	
hospital_id	uuid fk	Tenant
patient_id	uuid fk → patients	
doctor_id	uuid fk → doctors (nullable)	Assigned doctor
department_id	uuid fk → departments	
type	appointment_type	opd/follow_up/emergency
status	appointment_status	Lifecycle (Section 18)
source	lead_source	How it originated
is_walk_in	boolean	Walk-in vs online
preferred_date	date	Requested date
preferred_slot	text	Requested slot/range
confirmed_date	date (nullable)	After approval
confirmed_time	time (nullable)	After approval
token_number	int (nullable)	OPD token
problem	text	Symptoms/complaint
reject_reason	text (nullable)	If rejected
internal_notes	text (nullable)	Staff-only
doctor_notes	text (nullable)	Short consult note
follow_up_date	date (nullable)	If follow-up
assigned_by	uuid fk → users (nullable)	Receptionist who handled
created_at	timestamptz	
updated_at	timestamptz	

Indexes: hospital_id, status, doctor_id, department_id, preferred_date, confirmed_date, (hospital_id,status), (doctor_id,confirmed_date). RLS: staff of hospital full; doctor reads/updates only doctor_id = own; patient reads own via phone-matched patient.

appointment_slots (optional slot config)

Field	Type	Purpose / Notes
id	uuid pk	
hospital_id	uuid fk	Tenant
doctor_id	uuid fk	
weekday	int	0–6
start_time	time	
end_time	time	
slot_minutes	int	e.g., 15
capacity	int	Patients per slot (soft)
is_active	boolean	

Indexes: (hospital_id,doctor_id,weekday). RLS: public read (to show availability); Owner manages. (MVP can start with simple Morning/Evening ranges and adopt slots later.)

walk_ins (optional explicit log; or use appointments.is_walk_in)

Field	Type	Purpose / Notes
id	uuid pk	
hospital_id	uuid fk	Tenant
appointment_id	uuid fk → appointments	Link
added_by	uuid fk → users	Receptionist
created_at	timestampz	

RLS: staff of hospital. (Keeping is_walk_in on appointments is sufficient for MVP; this table is for explicit auditing.)

leads (CRM)

Field	Type	Purpose / Notes
id	uuid pk	
hospital_id	uuid fk	Tenant
patient_id	uuid fk (nullable)	If matched
appointment_id	uuid fk (nullable)	If booked
name	text	
phone	text	
city_area	text	

department_id	uuid fk (nullable)	Preferred
doctor_preference	uuid fk (nullable)	
source	lead_source	
status	lead_status	
appointment_date	date (nullable)	
follow_up_date	date (nullable)	Drives follow-up list
notes	text	
assigned_receptionist	uuid fk → users (nullable)	
created_at	timestampz	

Indexes: hospital_id, status, source, follow_up_date, (hospital_id,status), phone. RLS: staff of hospital; not patient-readable.

lead_sources (lookup; optional if using enum)

Field	Type	Purpose / Notes
id	uuid pk	
hospital_id	uuid fk (nullable)	Null = global default matches lead_source
key	text	
label	text	Display (bilingual)
is_active	boolean	

RLS: read by staff. (If using the enum, this table is just for labels/UX.)

notifications

Field	Type	Purpose / Notes
id	uuid pk	
hospital_id	uuid fk	Tenant
user_id	uuid fk (nullable)	Target user (null = role-wide)
target_role	user_role (nullable)	If role-targeted
type	text	new_request / reschedule / cancellation / follow_up_due
title	text	
body	text	

entity_type	text	e.g., 'appointment'
entity_id	uuid	
is_read	boolean	
created_at	timestampz	

Indexes: (hospital_id,user_id,is_read), created_at. RLS: user reads own / own-role within hospital.

analytics_events (lightweight event log)

Field	Type	Purpose / Notes
id	uuid pk	
hospital_id	uuid fk	Tenant
event_type	text	e.g., booking_submitted, call_click, whatsapp_click, status_check
entity_type	text (nullable)	
entity_id	uuid (nullable)	
metadata	jsonb	Extra (source, page, etc.)
created_at	timestampz	

Indexes: (hospital_id,event_type,created_at). RLS: insert allowed broadly (even anon for site events) but read only Owner/Super Admin. Use for source attribution and funnel.

settings (per-hospital key/value)

Field	Type	Purpose / Notes
id	uuid pk	
hospital_id	uuid fk	Tenant
key	text	e.g., 'arrival_instruction', 'default_language'
value	jsonb	
updated_at	timestampz	

Indexes: unique (hospital_id,key). RLS: Owner read/write; staff read.

hospital_pages (editable website content)

Field	Type	Purpose / Notes
id	uuid pk	

hospital_id	uuid fk	Tenant
page_key	text	'home_hero','about','faq','emergency', etc.
content	jsonb	Structured content blocks
updated_at	timestampz	

Indexes: unique (hospital_id,page_key). RLS: public read; Owner writes.

testimonials

Field	Type	Purpose / Notes
id	uuid pk	
hospital_id	uuid fk	Tenant
patient_name	text	
city_area	text	
rating	int	1–5
text	text	
photo_url	text (nullable)	
is_published	boolean	
created_at	timestampz	

Indexes: (hospital_id,is_published). RLS: public read published; Owner manages.

audit_logs

Field	Type	Purpose / Notes
id	uuid pk	
hospital_id	uuid fk	Tenant
actor_id	uuid fk → users (nullable)	Who did it
action	text	e.g., 'appointment.status_change'
entity_type	text	
entity_id	uuid	
before	jsonb (nullable)	Prior value
after	jsonb (nullable)	New value
created_at	timestampz	

Indexes: (hospital_id,entity_type,entity_id), created_at. RLS: Owner/Super Admin read; insert via triggers/service role only.

Entity relationship summary:

hospitals 1—* users
hospitals 1—* doctors *—1 departments
hospitals 1—* departments 1—* services
hospitals 1—* patients 1—* appointments *—1 doctors
appointments 1—0..1 leads appointments 1—0..1 walk_ins
hospitals 1—* leads hospitals 1—* notifications
hospitals 1—* hospital_pages / testimonials / settings / audit_logs /
analytics_events
users(doctor) 1—1 doctors (via users.doctor_id)

29. Row-Level Security Rules

Enable RLS on **every** table. Default deny; add explicit policies. Use a helper to read the caller's hospital and role from their users profile.

Helper functions (security definer):

-- Current user's hospital

create or replace function auth_hospital_id() returns uuid
language sql stable security **definer as** \$\$
 select hospital_id **from** public.users **where** id = auth.uid()
\$\$;

-- Current user's role

create or replace function auth_role() returns user_role
language sql stable security **definer as** \$\$
 select role **from** public.users **where** id = auth.uid()
\$\$;

-- Current user's doctor_id (for doctor logins)

create or replace function auth_doctor_id() returns uuid
language sql stable security **definer as** \$\$
 select doctor_id **from** public.users **where** id = auth.uid()
\$\$;

Core policy patterns:

-- Tenant isolation (apply to hospital-scoped staff tables)

create policy tenant_read **on** appointments **for select**
using (hospital_id = auth_hospital_id() **or** auth_role() = 'super_admin');

-- Staff write within own hospital (owner + receptionist)

create policy staff_write **on** appointments **for all**
using (hospital_id = auth_hospital_id() **and** auth_role() **in** ('owner','receptionist'))


```
with check ( hospital_id = auth_hospital_id() and auth_role() in
('owner','receptionist') );
```

-- Doctor: only own appointments, limited update

```
create policy doctor_read_own on appointments for select
using ( hospital_id = auth_hospital_id() and auth_role() = 'doctor' and doctor_id =
auth_doctor_id() );
```

```
create policy doctor_update_own on appointments for update
using ( auth_role() = 'doctor' and doctor_id = auth_doctor_id() )
with check ( doctor_id = auth_doctor_id() );
```

-- (App layer restricts doctor updates to notes/status/follow_up fields.)

-- Public website read (active records only) via anon role

```
create policy public_read_doctors on doctors for select
using ( is_active = true ); -- exposed via anon key on public pages
```

Per-role intent (summary):

Table	Super Admin	Owner	Receptionist	Doctor	Patient (anon/ph one)	Public (anon)
hospitals	all	read own	read own	read own	—	read public fields (active)
users	all	manage own hospital	read self	read self	—	—
doctors	all	manage own	read own	read self	—	read active
departments/ services	all	manage own	read own	read own	—	read active
patients	all	rw own hospital	rw own hospital	limited via appt join	own (phone match)	—
appointments	all	rw own hospital	rw own hospital	own only (limited)	own (phone match, via RPC)	insert request (RPC)
leads	all	rw own hospital	rw own hospital	—	—	—
notifications	all	own	own/role	own	—	—

analytics_events	read own	read own	—	—	—	insert (site events)
hospital_pages/testimonials	all	manage own	read	read	—	read published
audit_logs	read	read own	—	—	—	—

Important RLS rules:

- **Patient booking from public site** should go through a **SECURITY DEFINER RPC** (e.g., `create_appointment_request`) rather than direct anon INSERT, so validation, tenant binding, patient-dedup, and lead creation happen server-side and anon can't write arbitrary rows.
 - **Patient status lookup** also via an RPC (`get_patient_status(phone, hospital_id[, ref])`) that returns only that patient's safe fields — never a raw table select for anon.
 - **Doctor sees limited patient info**: enforce by exposing doctor reads through a **view** that selects only name, age, gender, problem (no address/phone unless required), plus RLS on the base table.
 - **Service role** (server-side only) used for emails, audit inserts, cross-tenant super-admin ops — never shipped to the client.
 - Never expose the service key to the browser; the public site uses the anon key with strict RLS + RPCs.
-

30. API / Backend Logic

Prefer **Supabase client + Postgres RPCs + a few Next.js Route Handlers / Server Actions**. Avoid a separate backend service in MVP.

Database RPCs (Postgres functions, security definer where noted):

- `create_appointment_request(payload jsonb)` → validates, binds `hospital_id`, upserts patient by (`hospital_id`, `phone`), inserts appointments (pending), inserts leads (website), inserts `analytics_event(booking_submitted)`, returns reference id. (*definer; callable by anon on public site.*)
- `add_walk_in(payload jsonb)` → staff-only; upsert patient, insert appointment (arrived, `is_walk_in`), insert lead (`walk_in`), issue token.
- `set_appointment_status(appointment_id, new_status, meta jsonb)` → staff/doctor (role-checked); validates transition, updates appointment, syncs linked lead, writes audit log + notification.
- `assign_appointment(appointment_id, doctor_id, confirmed_date, confirmed_time)` → staff-only; approve + assign.
- `issue_token(appointment_id)` → computes next token for (`hospital_id`, `doctor_id`, `date`).

- `get_patient_status(phone, hospital_id, ref default null)` – definer; returns safe status rows for that patient only.
- `request_reschedule_or_cancel(phone, appointment_id, action, meta)` – definer; creates a request + notification (does not directly mutate confirmed schedule).
- Analytics: `analytics_overview(hospital_id, from, to)`, `analytics_by_doctor(...)`, `analytics_by_department(...)`, `analytics_by_source(...)`, `analytics_timeseries(...)` → return aggregated JSON for the dashboard (Owner only via RLS/role check).

Next.js Route Handlers / Server Actions:

- `POST /api/booking` – thin wrapper calling `create_appointment_request` (rate-limit, honeypot, server-side validation with Zod).
- `POST /api/notify` – triggers email (Edge Function/Resend) on new request; abstracted `notify(event, payload)`.
- Server Actions for admin CRUD (doctors, departments, services, content) using the user's session (RLS enforced).
- Auth middleware: protect `/admin`, `/reception`, `/doctor`, `/super`; redirect by role after login.

Backend rules:

- All writes from public/anon go through definer RPCs (never raw anon inserts).
 - All status transitions validated against the state machine (Section 18) server-side.
 - Every sensitive mutation writes an `audit_logs` row (via trigger or inside the RPC).
 - Lead state is kept in sync with appointment state inside `set_appointment_status` (single source of truth).
 - Use Postgres triggers for `updated_at` and for auto-creating audit entries on appointments/hospital_pages updates.
 - Realtime (optional MVP+): subscribe receptionist dashboard to appointments inserts/updates for live request inbox; otherwise poll every ~20–30s.
-

31. Frontend Component Structure

Next.js **App Router**, TypeScript, Tailwind, shadcn/ui, React Hook Form + Zod, Recharts, Supabase SSR. Single multi-tenant app; tenant resolved by subdomain or path.

Folder structure:

```

/app
  /(public)                # marketing/public website (anon)
  /page.tsx                # Home
  /about/page.tsx
  /doctors/page.tsx
  /doctors/[slug]/page.tsx
  /departments/page.tsx
  /departments/[slug]/page.tsx
  /services/page.tsx

```

/book/page.tsx	# Appointment request form
/emergency/page.tsx	
/location/page.tsx	
/contact/page.tsx	
/faq/page.tsx	
/reviews/page.tsx	
/status/page.tsx	# Patient status (phone lookup)
/layout.tsx	# public header/footer, sticky CTAs
/(auth)	
/login/page.tsx	
/admin	# Owner/Admin (protected)
/layout.tsx	# admin shell + nav + role guard
/page.tsx	# KPI snapshot
/profile/page.tsx	
/doctors/page.tsx	
/departments/page.tsx	
/services/page.tsx	
/timings/page.tsx	
/staff/page.tsx	
/appointments/page.tsx	
/leads/page.tsx	
/analytics/page.tsx	
/content/page.tsx	
/settings/page.tsx	
/reception	# Receptionist (protected)
/layout.tsx	
/page.tsx	# Requests + today + queue
/walk-in/page.tsx	
/leads/page.tsx	
/doctor	# Doctor (protected)
/layout.tsx	
/page.tsx	# Today's queue
/super	# Super Admin (protected)
/layout.tsx	
/page.tsx	
/hospitals/page.tsx	
/api	
/booking/route.ts	
/notify/route.ts	
/components	
/ui	# shadcn primitives
/website	# Hero, DoctorCard, DepartmentCard, CTASlicky,
WhatsAppButton, Testimonials, MapEmbed, TrustBadges, OpdTImings	
/booking	# BookingForm, SlotPicker, ConfirmationCard
/reception	# RequestCard, StatusBadge, ApproveDialog,
RescheduleDialog, WalkInForm, QueueList, TokenBadge, Filters	
/doctor	# DoctorQueue, PatientRow, ConsultNote
/patient	# StatusLookup, StatusCard, RescheduleRequest
/crm	# LeadTable, LeadFilters, LeadStatusSelect,
AddLeadDialog, SourceBadge	

/analytics	# KpiCard, BarByDoctor, BarByDepartment, SourcePie, TrendLine, DateRangeToggle
/admin	# CrudTable, EntityForm, NavSidebar, ContentEditor
/common	# LanguageToggle, ConfirmDialog, EmptyState, Loading, ErrorState
/lib	
/supabase	# client.ts (browser), server.ts (SSR), middleware.ts, admin.ts (service role, server-only)
/validation	# zod schemas (booking, lead, doctor, department, ...)
/rpc	# typed wrappers for RPC calls
/auth	# session, role-guard helpers
/i18n	# hi / en / hinglish strings
/utils	# formatters (phone, date), token, constants
/analytics	# event tracking helper (analytics_events)
/types	# generated Supabase types + domain types
/middleware.ts	# route protection + tenant resolution

Component rules:

- Server Components for data fetching (public pages, lists); Client Components only where interactive (forms, dialogs, filters, charts).
 - Generate and use Supabase **TypeScript types** (types/); no any.
 - All forms: React Hook Form + Zod; shared schemas in /lib/validation reused on client + server (route handlers/RPC validation).
 - Reusable CrudTable + EntityForm drive all admin management screens (doctors, departments, services).
 - A single StatusBadge and status color map shared across reception/doctor/patient.
 - Bilingual labels come from /lib/i18n; never hardcode user-facing strings in components.
 - Charts (Recharts) wrapped in /components/analytics; keep them simple and mobile-legible.
-

32. Mobile Responsiveness Requirements

Mobile is the **primary** experience for patients, doctors, and owners. Desktop is primary only for the receptionist.

- **Mobile-first CSS** (Tailwind): design at 360–414px width first, scale up.
- **Touch targets** ≥ 44px; primary CTAs full-width on mobile.
- **Sticky bottom action bar** on the public site (Call · WhatsApp · Book · Directions).
- **Booking form** usable one-handed: numeric keypad for phone/age, native date picker, minimal typing, no horizontal scroll.
- **Doctor dashboard** fully usable on a phone: one patient per row, big "Completed" button.
- **Owner analytics** stacks to single column on mobile; charts remain legible (no tiny dense charts).

- **Reception dashboard** responsive down to tablet/phone for counter staff using a phone, but optimized for desktop.
 - **Tables** → **cards** on small screens (no horizontal scrolling of critical data).
 - Test on real low-end Android (small screens, slow CPU); avoid heavy animations.
-

33. Hindi / Hinglish UX Requirements

Language is a trust and adoption lever, not a nice-to-have.

- **Patient-facing surfaces** (website, booking form, status page, confirmation) support **Hindi/Hinglish** with an easy **language toggle** (हिंदी / English). Default follows `hospitals.default_language`.
 - **Receptionist labels**: key actions and statuses available in Hinglish (e.g., "Naya Request", "Approve karein", "Reschedule", "Aa gaya / Arrived", "Ho gaya / Completed").
 - **Dashboards** (doctor, owner, admin) can stay in **simple English** — short, plain words; avoid jargon.
 - **Status messages** bilingual: e.g., "Aapki appointment confirm ho gayi hai. Kripya 15 minute pehle aayein. / Your appointment is confirmed. Please arrive 15 minutes early."
 - **WhatsApp prefilled text** in Hinglish: "Namaste, mujhe [Department] me appointment chahiye."
 - All strings centralized in `/lib/i18n` (hi, en, hinglish); never hardcode user-facing copy.
 - Keep sentences short and concrete; avoid English medical/enterprise jargon on patient screens.
 - Numerals, dates, and phone formats in familiar Indian formats (DD MMM, +91 / 10-digit).
-

34. Local SEO Requirements

The website should rank for local intent ("best [specialty] hospital in [city]", "[department] doctor near me").

- **Per-hospital metadata**: title/description templates with hospital name + city + specialty; unique per page.
- **Semantic structure**: one H1 per page (hospital + city + specialty), clean heading hierarchy.
- **Schema.org structured data**: MedicalOrganization / Hospital, Physician (per doctor), MedicalClinic, LocalBusiness with address, geo, opening hours, phone; BreadcrumbList; FAQPage on FAQ; AggregateRating/Review on testimonials.
- **NAP consistency** (Name, Address, Phone) matching Google Business Profile.
- **City + department landing pages** (`/departments/[slug]`, doctor pages) for long-tail local queries.

- **Performance = SEO:** fast LCP, server-rendered content, real text (not images of text).
 - **Sitemap.xml + robots.txt**, canonical URLs, OpenGraph/Twitter cards for shareable links.
 - **Image alt text** with hospital/city/specialty; descriptive file names.
 - Encourage Google reviews (link out) and surface ratings on-site.
 - Mobile-friendliness and HTTPS (Vercel) as baseline ranking factors.
-

35. Google Maps and Local Trust Elements

Local trust converts in Tier 2/3 markets. Build it in:

- **Embedded Google Map** on Home, Location, and Contact; tappable "**Get Directions**" (opens Google Maps app).
 - **Google Business Profile** linkage: show rating + review count; deep-link to leave a review.
 - **Trust badges:** years of service, number of doctors/departments, "Govt. Registered", accreditations, 24x7 emergency.
 - **Real doctor photos + qualifications** (not stock); real patient testimonials with names/area.
 - **Clear OPD timings** and **emergency number** visible site-wide.
 - **One-tap Call** and **WhatsApp** everywhere — the strongest trust + conversion signal locally.
 - **Address with landmark** (Tier 2/3 patients navigate by landmarks, not pin codes).
 - Consistent phone number across site, Maps, and WhatsApp.
-

36. Performance Requirements

Patients are on phones and patchy 4G; staff need snappy dashboards.

- **Public pages server-rendered** (Next.js RSC) and statically cached where possible (ISR for doctor/department pages).
- **Core Web Vitals targets:** LCP < 2.5s on 4G, CLS < 0.1, INP < 200ms.
- **Lightweight bundles:** client JS only where needed; avoid heavy libraries on public pages; lazy-load charts (Recharts) and dialogs.
- **Optimized images:** next/image, correct sizes, WebP, lazy loading; compress doctor photos in Storage.
- **Fast booking:** form interactive immediately; submit returns within ~1s perceived (optimistic confirmation).
- **Dashboards:** paginate/limit queries; index-backed filters; avoid N+1 (use joins/views/RPCs).
- **Caching:** cache public content; revalidate on admin content edits.
- **Low-bandwidth mode:** minimal fonts, system font stack option, no autoplaying media.

- Deploy on **Vercel** (edge/CDN) with Supabase region close to India (e.g., Mumbai/Singapore) for low latency.
-

37. Security and Privacy Considerations

Practical but serious — this is patient data.

- **Role-based access** enforced server-side via **Supabase RLS** on every table (default deny).
 - **Multi-tenant isolation**: every query scoped by `hospital_id`; verified by RLS, not just UI.
 - **Least-privilege for doctors**: doctors see only their own patients and only necessary fields (via restricted view).
 - **No public exposure of private patient data**: anon site can only create requests and read its own status via definer RPCs; never raw selects on patients/appointments.
 - **Service role key server-only**: never shipped to the browser; used only in server actions/Edge Functions.
 - **Audit logs** for sensitive updates (status changes, assignments, content, account changes).
 - **Secure admin routes**: middleware + server session checks; redirect unauthenticated/unauthorized.
 - **Input validation** everywhere (Zod on client + server; RPC-side checks).
 - **Rate limiting + honeypot** on public booking/contact to limit spam/abuse.
 - **PII minimization**: collect only what's needed; mask phone where not required; restrict patient phone visibility by role.
 - **Transport security**: HTTPS only (Vercel), secure cookies, Supabase SSR session handling.
 - **Data lifecycle**: define retention for leads/appointments; soft-disable hospitals rather than hard-delete; back up via Supabase.
 - **Compliance posture**: align with India's DPDP Act principles (consent, purpose limitation, data minimization); add a privacy policy page; obtain consent on booking ("I agree to be contacted").
-

38. MVP Development Roadmap

Build philosophy: do not overbuild. Ship the smallest thing that delivers the core promise, then iterate. When in doubt, choose the simpler implementation (Morning/Evening ranges before full slot grids; polling before realtime; email before WhatsApp).

MVP includes: website, appointment request form, receptionist dashboard, doctor dashboard, patient status view, lead CRM, owner analytics, admin management, basic OPD/walk-in entry, basic notifications (in-app + email).

MVP excludes (Section 11): EHR, prescriptions, lab reports, payments, teleconsultation, mobile apps, AI assistant, insurance, billing, multi-branch/enterprise.

Milestones:

1. **Foundation** — Next.js + Supabase + Auth + multi-tenant + schema + RLS.
 2. **Public website + booking** — pages, CTAs, booking RPC, lead creation.
 3. **Receptionist + appointments** — request inbox, approve/reschedule/reject, statuses, walk-in, token.
 4. **Doctor dashboard** — today's queue, notes, complete, follow-up.
 5. **Patient status** — phone lookup, status, reschedule/cancel request.
 6. **CRM + Analytics** — lead table + owner KPIs/charts.
 7. **Admin panel + notifications + polish** — management screens, email + in-app notifications, i18n, SEO, performance.
-

39. Phase-Wise Build Plan for Claude Code

Each phase is independently demoable. Build in order; don't start a phase before the previous is acceptance-complete (Section 40).

Phase 0 — Project setup (foundation)

- Init Next.js (App Router, TS) + Tailwind + shadcn/ui; ESLint/Prettier.
- Connect Supabase; configure SSR auth (cookies); env management.
- Create enums + all tables (Section 28); enable RLS + helper functions + base policies (Section 29).
- Generate Supabase TypeScript types; set up `/lib/supabase`, `/lib/i18n`, `/lib/validation`.
- Multi-tenant resolution (subdomain/path → hospital_id); seed one demo hospital with doctors/departments/services.
- **Demo:** seeded data visible; auth login for each role routes correctly.

Phase 1 — Public website + booking (Reels 1, 2, 8)

- Build public pages (Section 15) with real-content components, sticky CTAs, WhatsApp, map, trust badges.
- Implement `/book` form (RHF + Zod) — `create_appointment_request` RPC — creates appointment (pending) + patient + lead + analytics event.
- Confirmation screen with status link; Call/WhatsApp fallback.
- Local SEO (metadata, schema, sitemap) + performance pass.
- **Demo:** patient books a request in under 30 seconds; lead appears in DB.

Phase 2 — Receptionist dashboard + OPD/walk-in (Reels 3, partly 10)

- Request inbox (pending), Today view, filters/search.
- Approve / Reschedule / Reject (with reason); assign doctor + slot; status state machine via `set_appointment_status`.

- Add walk-in (add_walk_in); token issue; queue list; arrived → in_consultation → completed.
- In-app notifications + audit logs on changes.
- **Demo:** request flows from patient → receptionist → approved; walk-in added and queued.

Phase 3 — Doctor dashboard (Reel 4)

- Today/Tomorrow/Pending/Arrived/Completed tabs; RLS "own appointments only".
- Patient basics + problem; short note; mark completed; request follow-up; date filter.
- **Demo:** doctor opens phone, sees today's queue, completes a consult, sets a follow-up.

Phase 4 — Patient status view (Reel 5)

- /status phone lookup via get_patient_status RPC; status card with date/time/doctor/location/contact.
- Reschedule/cancel **request** creates a receptionist task + notification.
- Bilingual arrival instructions.
- **Demo:** patient checks status without calling; requests a reschedule.

Phase 5 — Lead CRM (Reel 6)

- Lead table + filters (source/status/department/date/assignee); inline status change; click-to-call; add-lead (manual channels); follow-up list.
- Auto-sync lead status with appointment lifecycle.
- **Demo:** every inquiry (web, walk-in, manual phone/WhatsApp) visible with status + follow-up.

Phase 6 — Owner analytics (Reel 7)

- KPI cards + breakdowns via analytics RPCs/views; date-range toggle; mobile layout.
- **Demo:** owner sees today's requests, conversions, source-wise leads, busiest doctor/department, growth.

Phase 7 — Admin panel + notifications + polish (Reels 9, 10)

- Admin CRUD (profile, doctors, departments, services, timings, staff, content, settings).
 - Email notification on new request (Edge Function + Resend); finalize in-app notifications.
 - Super Admin tenant management.
 - i18n completeness, SEO/perf/security hardening, edge cases (Section 41), acceptance pass (Section 40).
 - **Demo:** full ecosystem end-to-end (Reel 10).
-

40. Acceptance Criteria

Definition of done per module. A phase is complete only when its criteria pass.

Website & Booking

- All public pages render server-side, mobile-first, with visible Call/WhatsApp/Book/Directions.
- A new visitor can submit a complete appointment request in **< 30 seconds**; invalid input shows inline bilingual errors.
- Submit creates exactly one appointment (pending) + patient (deduped by phone) + lead (source website) + analytics event; confirmation + status link shown.
- Lighthouse mobile: Performance ≥ 90 , SEO ≥ 95 ; valid structured data.

Receptionist

- New requests appear within ≤ 30 s (poll/realtime) and are searchable by name/phone instantly.
- Approve/Reschedule/Reject doable in ≤ 2 taps; status transitions follow the state machine; invalid transitions blocked.
- Walk-in added in ≤ 20 s and appears in queue + on the doctor's list; token issued correctly (per doctor/day).
- Every status change writes an audit log and updates the linked lead.

Doctor

- Doctor sees only their own appointments (verified by RLS test with another doctor's token).
- Today's queue ordered by token; mark-completed and follow-up work; no PII beyond allowed fields.

Patient status

- Phone lookup returns only that patient's appointments (no cross-patient leakage; verified).
- Reschedule/cancel creates a receptionist-visible request without mutating the confirmed schedule directly.

Lead CRM

- 100% of inquiries (web + walk-in + manual) are stored as leads; filters and follow-up list work; status auto-syncs with appointments.

Owner analytics

- KPI numbers reconcile with raw table counts for a test dataset; date-range toggle correct; charts legible on mobile.

Admin

- All CRUD scoped to the owner's hospital; cannot read/write another hospital's data (RLS-verified); content edits reflect on the public site.

Cross-cutting

- RLS enabled on every table; anon cannot read patients/appointments directly; service key not in client bundle.
 - Language toggle switches patient-facing copy; no hardcoded user-facing strings.
-

41. Edge Cases

Handle these explicitly (and add tests where noted):

- **Duplicate phone / repeat patient:** match existing patient by (hospital_id, phone); don't create duplicates; merge new request into same patient.
- **Same patient books multiple times:** allow, but flag possible duplicate request to receptionist.
- **Doctor unavailable / on leave after booking:** receptionist reschedules or reassigns; patient status updates; reason captured.
- **OPD timing / holiday changes:** bookings on holidays blocked or flagged; admin can set holidays; existing affected appointments surfaced for rescheduling.
- **Walk-in for a fully-booked doctor:** allowed (walk-ins aren't slot-limited in MVP) but added to queue with token; owner sees load.
- **No doctor selected ("Any available"):** receptionist assigns at approval.
- **Patient enters wrong/invalid phone:** validation blocks submit; status lookup fails gracefully with Call fallback.
- **Status lookup with no/expired appointment:** friendly empty state + Call/WhatsApp.
- **Reschedule/cancel request after patient already arrived:** receptionist sees it but queue state takes precedence; resolve manually.
- **No-show handling:** mark no_show; lead becomes follow-up/lost; counted in analytics.
- **Past-date or far-future date:** block past; cap future window (configurable).
- **Concurrent receptionists acting on the same request:** last-write-wins with audit trail; optionally lock/disable buttons on status change (optimistic UI + refetch).
- **Token continuity across the day / midnight rollover:** token resets per (hospital_id, doctor_id, date).
- **Multi-tenant leakage:** a user from hospital A must never see hospital B's data — covered by RLS tests.
- **Doctor login not linked to a doctor record:** guard and show setup message; admin links users.doctor_id.
- **Spam/bot bookings:** rate limit + honeypot; suspicious entries flagged, not auto-trusted.
- **Network failure mid-submit:** idempotent booking (dedupe by phone + timestamp window) to avoid double requests.
- **Low bandwidth/timeout:** retrievable actions; clear loading/error states; no data loss on retry.

- **Inactive/disabled doctor or department:** hidden from booking but historical appointments still viewable.
 - **Language fallback:** missing translation falls back to English, never blank.
-

42. Future Scalability Plan

The MVP is built so these add on without re-architecting:

- **Communication:** WhatsApp bot + WhatsApp/SMS notifications and reminders; automated follow-up reminders; missed-call → CRM (slot into the notify() abstraction).
- **Payments:** UPI / online payments / payment gateway for advance booking fees; receipts.
- **Clinical:** teleconsultation (video), digital prescriptions, lab report upload, patient medical history (new tables, gated behind clinical role/permissions).
- **Apps:** doctor mobile app and patient mobile app (reuse the same Supabase backend/RPCs).
- **Multi-branch / enterprise:** branch dimension on tenants; org → branches → doctors; consolidated analytics.
- **Growth tooling:** review collection system, referral tracking, marketing source attribution, campaign UTM capture into analytics_events.
- **Operations:** staff attendance, inventory, pharmacy module, basic billing.
- **AI:** AI receptionist assistant (auto-draft replies, triage, summarize complaints), smart scheduling suggestions.
- **Scale-out:** read replicas / materialized views for analytics; partition large tables by hospital_id/date; queue-based notifications; per-tenant custom domains.

Architectural enablers already in MVP: multi-tenant hospital_id, RPC/service abstraction, notify(event, payload) indirection, analytics_events for attribution, clean role model, generated types.

43. Sales Demo Flow for Hospital Owners

A 5-minute, outcome-led demo for a Kanpur/Lucknow-style owner. Lead with pain and money, not features. Use a seeded demo hospital on a phone.

1. **The problem (10s):** "Aapki reception ka phone busy rehta hai, calls miss hoti hain, patient doosre hospital chala jaata hai. Register aur memory pe sab chalta hai." Show a typical old brochure-style website that can't book.
2. **Patient books in 30 seconds (live):** On a phone, open the hospital site → tap **Book Appointment** → pick department/doctor/date → enter name + phone + problem → submit. "Bas itna hi — 30 second me request aa gayi."

3. **Receptionist approval dashboard:** Switch to reception screen — the request is already there. **Approve** in 2 taps, assign doctor + slot. Show search by phone and the walk-in button. "Ek hi screen pe sab — calls, walk-in, sab yahin."
4. **Doctor dashboard:** Open the doctor's phone view — today's patients in order with their complaint. Mark one **Completed**, set a **Follow-up**. "Doctor ko bas aaj ki list dikhti hai, aur kuch nahi."
5. **Patient status view:** Patient opens /status, enters phone — sees "Confirmed, Dr. X, 11:30 AM, 15 min pehle aayein." "Ab patient baar-baar call nahi karega."
6. **Lead CRM:** Show every inquiry — website, WhatsApp, walk-in — with status and follow-up date. "Ek bhi patient ab miss nahi hoga. Jo nahi aaye, woh bhi yahan dikhte hain."
7. **Owner analytics:** Owner's phone — today's requests, approved, completed, no-shows, source-wise leads, busiest doctor/department, weekly growth. "Aapko roz pata chalega hospital kaisa chal raha hai."
8. **The outcome (close):** "Kam missed calls, behtar patient experience, zyada appointments, aur poora control — aapke phone pe. Monthly subscription, aaj se shuru." Offer to set up their hospital (profile + doctors) on the spot.

Demo rules: keep it on a phone, keep it bilingual, never show code/DB, always tie each screen to money or fewer lost patients.

44. Marketing Angle

Core message: "Your hospital is losing patients to missed calls and a paper register. This is your hospital's modern front desk — book appointments, never lose an inquiry, and see your growth, all from your phone."

Positioning pillars:

- **Built for Bharat hospitals,** not imported enterprise software. Hindi/Hinglish, mobile-first, simple.
- **Outcomes over features:** fewer missed calls, more appointments, full visibility.
- **Affordable + fast to start:** live in a week, low monthly price, no IT team needed.
- **Made for non-technical staff:** if your receptionist can use WhatsApp, she can use this.

Proof & trust: before/after of an old brochure site vs. a booking machine; live 30-second booking; owner-analytics screenshot; testimonials from early hospitals.

Channels: Instagram/YouTube reels (Section 45), local hospital owner WhatsApp groups, direct outreach in target cities, referrals from onboarded hospitals.

Offer: free setup + first-month trial; "we'll build your hospital's new website and front-desk system this week."

45. 10-Part Reel Series Alignment

The product maps 1:1 to a 10-part reel series — "Hospital Growth & Automation Solutions" / "How Modern Hospitals Will Work in 2026" — so every feature has a story and a demo.

Part	Title	Problem	Product moment to show
1	Why Most Hospital Websites Fail	Sites are brochures, don't book	Old site vs. conversion-focused site (Module 1)
2	Smart Appointment Booking	Reception busy, calls missed	Live 30-second booking (Module 2)
3	Receptionist Approval System	Calls/WhatsApp/registers = chaos	One dashboard: approve/reject/reschedule (Module 3)
4	Doctor Dashboard	Doctors lack appointment visibility	Today/tomorrow/pending/arrived (Module 4)
5	Patient Dashboard	Patients call repeatedly for status	Phone-based status check (Module 5)
6	Lead Management CRM	Inquiries get lost	Every inquiry stored with status + follow-up (Module 6)
7	Hospital Analytics Dashboard	Owner doesn't know performance	KPIs, source-wise leads, growth (Module 7)
8	Modern Hospital Website	Old sites don't convert or build trust	Premium, fast, mobile, local-SEO site (Module 1)
9	Future of Hospitals	What's next	WhatsApp bot, payments, teleconsult, lab reports, apps (Section 42)
10	Complete Hospital Growth Ecosystem	Tie it together	Full flow: Patient → Website → Appointment → Receptionist → Doctor → CRM → Analytics → Growth

Each reel ends with the same CTA: "Want this for your hospital? DM us." Build order (Section 39) intentionally follows this sequence so each phase ships a demoable reel.

46. Final Developer Checklist

Foundation

- ☐ Next.js (App Router, TS) + Tailwind + shadcn/ui set up; lint/format configured.
- ☐ Supabase project; SSR auth (cookies); env vars; service key server-only.
- ☐ All enums + tables created (Section 28); updated_at triggers.
- ☐ RLS enabled on every table; helper functions + policies (Section 29); default deny.
- ☐ Multi-tenant resolution (subdomain/path → hospital_id); demo hospital seeded.
- ☐ Supabase TypeScript types generated; no any.

Public site + booking

- ☐ All public pages (Section 15) built, mobile-first, server-rendered.
- ☐ Sticky CTAs: Call / WhatsApp / Book / Directions on every page.
- ☐ Booking form (RHF + Zod) → create_appointment_request RPC → appointment + patient + lead + event.
- ☐ Confirmation + status link; spam controls (rate limit + honeypot); consent checkbox.
- ☐ Local SEO (metadata, schema.org, sitemap, robots); Lighthouse mobile Perf ≥ 90 / SEO ≥ 95.

Reception + OPD

- ☐ Request inbox + Today view; instant search by name/phone; filters.
- ☐ Approve / Reschedule / Reject (reason); assign doctor + slot; state machine enforced.
- ☐ Walk-in add; token per (hospital, doctor, day); queue: arrived → in_consultation → completed.
- ☐ Audit log + lead sync on every status change; in-app notifications.

Doctor

- ☐ Today/Tomorrow/Pending/Arrived/Completed; RLS own-only (cross-doctor test passes).
- ☐ Patient basics + problem; short note; complete; follow-up; date filter; limited PII.

Patient status

- ☐ Phone lookup via get_patient_status RPC (no cross-patient leakage).
- ☐ Reschedule/cancel request → receptionist notification; bilingual arrival instructions.

CRM + analytics

- ☐ Lead table + filters + follow-up list; manual add-lead; click-to-call; status auto-sync.
- ☐ Owner KPIs + breakdowns (doctor/department/source/timeseries) via RPCs; numbers reconcile; mobile layout.

Admin + polish

- ☐ Admin CRUD: profile, doctors, departments, services, timings, staff, content, settings (RLS-scoped).
 - ☐ Super Admin tenant management.
 - ☐ Email on new request (Edge Function + Resend); notify() abstraction in place.
 - ☐ i18n complete (hi/en/hinglish); language toggle; no hardcoded user-facing strings.
 - ☐ Edge cases (Section 41) handled; acceptance criteria (Section 40) pass.
 - ☐ Security pass: anon cannot read patients/appointments; service key absent from client bundle; HTTPS; audit logs working.
 - ☐ Performance pass: Core Web Vitals targets met; images optimized; charts/dialogs lazy-loaded.
 - ☐ Deployed on Vercel; Supabase region near India; backups enabled.
-

End of PRD. Build MVP-first, in phase order, against the acceptance criteria. When in doubt, ship the simpler version.